

Library Management System

Project approach report covering ER design, operation implementation, code excerpts, screenshots, challenges, and testing evidence.

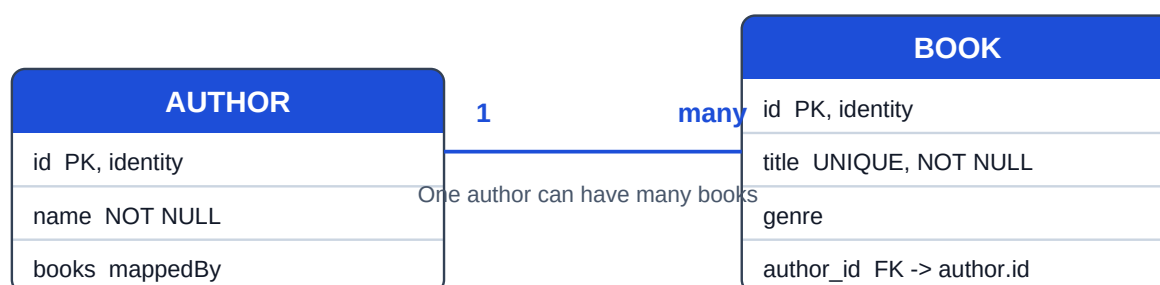
1. Project Overview

This project is a Spring Boot MVC web application for maintaining a simple library book catalog. It uses Spring Data JPA with Hibernate for persistence, an H2 in-memory database for development, JSP views for the user interface, and Maven for building and testing.

- **Backend:** Spring Boot 4.0.6, Spring MVC, Spring Data JPA, Hibernate.
- **Database:** H2 in-memory database at `jdbc:h2:mem:librarydb`.
- **Frontend:** JSP files under `src/main/webapp/WEB-INF/jsp` with embedded CSS.
- **Main feature set:** list books with authors, add a book, edit an existing book, and show a clear error for duplicate or empty book titles.

2. Entity Relationship Design

The database model is intentionally compact: authors are stored separately from books, and every book must belong to one author. This avoids repeating author names for every book and gives the project a clean one-to-many relationship.



Entity	Important Fields	Constraints / Relationship
Author	id, name, books	id is the primary key. name is required. One Author has many Book records.
Book	id, title, genre, author	id is the primary key. title is required and unique. author_id is a required foreign key.

Entity mapping: src/main/java/com/example/library/entity/Author.java

```
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL)
    private List<Book> books;
}
```

Entity mapping: src/main/java/com/example/library/entity/Book.java

```
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String title;

    private String genre;

    @ManyToOne
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3. Implementation Details for Each Operation

3.1 Read Operation: List Books with Authors

The list page is served by GET /books. The controller asks the service for all books, the service delegates to a custom repository query, and the JSP renders each book with its nested author name.

Controller + service + repository flow

```
@GetMapping
public String listBooks(Model model) {
    model.addAttribute("books", libraryService.getAllBooksWithAuthors());
    return "list-books";
}

public List<Book> getAllBooksWithAuthors() {
    return bookRepository.findAllBooksWithAuthorsInnerJoin();
}

@Query("SELECT b FROM Book b INNER JOIN FETCH b.author")
List<Book> findAllBooksWithAuthorsInnerJoin();
```

Library Book Catalog				
Add New Book				
ID	Title	Genre	Author	Actions
1	1984	Dystopian	George Orwell	Edit
2	Harry Potter and the Sorcerer's Stone	Fantasy	J.K. Rowling	Edit
3	The Fellowship of the Ring	High Fantasy	J.R.R. Tolkien	Edit
4	Murder on the Orient Express	Mystery	Agatha Christie	Edit
5	The Shining	Horror	Stephen King	Edit
6	Foundation	Science Fiction	Isaac Asimov	Edit
7	Pride and Prejudice	Romance	Jane Austen	Edit
8	The Great Gatsby	Tragedy	F. Scott Fitzgerald	Edit
9	The Hound of the Baskervilles	Detective Fiction	Arthur Conan Doyle	Edit
10	Frankenstein	Gothic Science Fiction	Mary Shelley	Edit

Screenshot 1: Book catalog rendered from seeded H2 data using the inner join query.

3.2 Create Operation: Show Add Book Form

The add form is served by GET `/books/new`. The controller sends an empty Book object for Spring form binding and all authors for the author dropdown.

Controller code for opening the create form

```
@GetMapping("/new")
public String showAddForm(Model model) {
    model.addAttribute("book", new Book());
    model.addAttribute("authors", libraryService.getAllAuthors());
    return "book-form";
}
```

JSP form binding: `src/main/webapp/WEB-INF/jsp/book-form.jsp`

```
<form:form action="/books/save" modelAttribute="book" method="POST">
  <form:hidden path="id" />
  <form:input path="title" required="required" placeholder="Enter book title" />
  <form:input path="genre" required="required" placeholder="Enter genre" />
  <form:select path="author.id">
    <form:option value="" label="-- Select an Author --" />
    <c:forEach var="auth" items="${authors}">
      <form:option value="${auth.id}"${auth.name}</form:option>
    </c:forEach>
  </form:select>
</form:form>
```

Screenshot 2: Add New Book form with title, genre, and author dropdown.

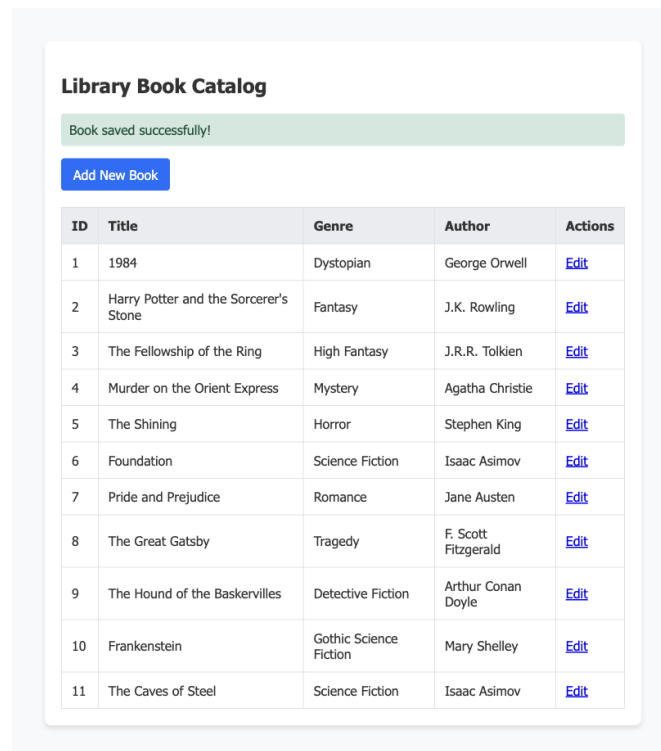
3.3 Save Operation: Insert a New Book

The same POST `/books/save` endpoint handles saving. When the submitted book has no id, Spring Data JPA treats `bookRepository.save(book)` as an insert. A flash message is used after redirect so the user sees confirmation on the list page.

POST handler and service save method

```
@PostMapping("/save")
public String saveBook(@ModelAttribute("book") Book book,
                      RedirectAttributes redirectAttributes) {
    try {
        libraryService.saveBook(book);
        redirectAttributes.addFlashAttribute("success", "Book saved successfully!");
    } catch (RuntimeException e) {
        redirectAttributes.addFlashAttribute("error", e.getMessage());
        return "redirect:/books/new";
    }
    return "redirect:/books";
}

public void saveBook(Book book) {
    bookRepository.save(book);
}
```



Screenshot 3: Successful create operation redirects back to the catalog with a success message.

3.4 Update Operation: Edit an Existing Book

The edit form is served by GET `/books/edit/{id}`. The controller loads the existing book and reuses the same JSP. Because the form includes a hidden id field, the save endpoint performs an update instead of an insert.

Controller code for opening the edit form

```
@GetMapping("/edit/{id}")
public String showEditForm(@PathVariable Long id, Model model) {
    model.addAttribute("book", libraryService.getBookById(id));
    model.addAttribute("authors", libraryService.getAllAuthors());
    return "book-form";
}

public Book getBookById(Long id) {
    return bookRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Book with ID " + id + " not found"));
}
```

Screenshot 4: The same form is reused for updating an existing book.

3.5 Data Integrity Handling

The `Book.title` column is both required and unique. If the database rejects a duplicate title, the service catches the Spring data exception and returns a friendly message to the form instead of exposing a raw SQL error.

Friendly integrity handling in `LibraryService`

```
public void saveBook(Book book) {
    try {
        bookRepository.save(book);
    } catch (DataIntegrityViolationException e) {
        throw new RuntimeException(
            "Data integrity violation: Book title must be unique and cannot be empty."
        );
    }
}
```

Add New Book

Data integrity violation: Book title must be unique and cannot be empty.

Title:

Genre:

Author:

Save Book **Cancel**

Screenshot 5: Duplicate title submission is rejected and shown as a clear validation message.

4. Database Initialization

The application uses `spring.jpa.hibernate.ddl-auto=update` so Hibernate creates or updates the schema from the entity classes. The `data.sql` file seeds 10 authors and 10 books.

`spring.jpa.defer-datasource-initialization=true` ensures the schema exists before seed data is inserted.

Configuration: `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
spring.jpa.defer-datasource-initialization=true
```

Seed data example: `src/main/resources/data.sql`

```
INSERT INTO author (name) VALUES ('George Orwell');
INSERT INTO author (name) VALUES ('J.K. Rowling');

INSERT INTO book (title, genre, author_id)
VALUES ('1984', 'Dystopian', 1);
INSERT INTO book (title, genre, author_id)
VALUES ('Harry Potter and the Sorcerer's Stone', 'Fantasy', 2);
```

5. Challenges Faced and How They Were Overcome

Challenge	How it was handled
Representing the author-book relationship cleanly	Separated Author and Book into two JPA entities. Book owns the foreign key with <code>@ManyToOne</code> and <code>@JoinColumn</code> , while Author exposes the inverse <code>@OneToMany</code> relationship.
Displaying author details without extra queries	Used an explicit JPQL INNER JOIN FETCH query so the list page receives books and authors in one query.
Binding a selected author from a JSP dropdown	Used Spring form binding with <code>path="author.id"</code> , allowing the selected author id to be bound into the Book object.
Showing validation errors clearly	Added a unique and not-null constraint on title, caught <code>DataIntegrityViolationException</code> in the service, and displayed a friendly flash error on the form.
Running JSPs in embedded Tomcat	Added Jasper and JSTL dependencies in <code>pom.xml</code> , then configured the MVC view prefix and suffix for <code>/WEB-INF/jsp/</code> .
Loading seed data reliably	Enabled deferred datasource initialization so <code>data.sql</code> runs after Hibernate creates the tables.

6. Testing and Verification

The project includes Spring Boot tests for context loading and service behavior. The service tests verify that seeded books are returned, book id lookup works, authors are loaded, and duplicate titles produce the expected friendly error message.

Test evidence: `src/test/java/com/example/library/service/LibraryServiceTest.java`

```
@Test
void getAllBooksWithAuthorsReturnsSeededBooks() {
    List<Book> books = libraryService.getAllBooksWithAuthors();
    assertFalse(books.isEmpty());
    assertTrue(books.stream().anyMatch(book -> "1984".equals(book.getTitle())));
    assertTrue(books.stream().allMatch(book -> book.getAuthor() != null));
}

@Test
void saveBookThrowsFriendlyMessageOnIntegrityViolation() {
    Author author = libraryService.getAllAuthors().get(0);
    Book duplicate = new Book("1984", "Dystopian", author);
    RuntimeException exception = assertThrows(
        RuntimeException.class,
        () -> libraryService.saveBook(duplicate)
    );
    assertEquals(
        "Data integrity violation: Book title must be unique and cannot be empty.",
        exception.getMessage()
    );
}
```

Verification run: `./mvnw test` completed successfully with **4 tests run, 0 failures, 0 errors, 0 skipped**.

7. Conclusion

The project demonstrates a complete database-backed MVC flow for a library catalog: normalized entity design, Spring Data repository access, service-layer validation handling, JSP form binding, and visible user feedback. The design can be extended naturally with delete operations, search/filtering, server-side validation annotations, and a persistent production database.